ML - Machine Learning

Regression

Riccardo Berta

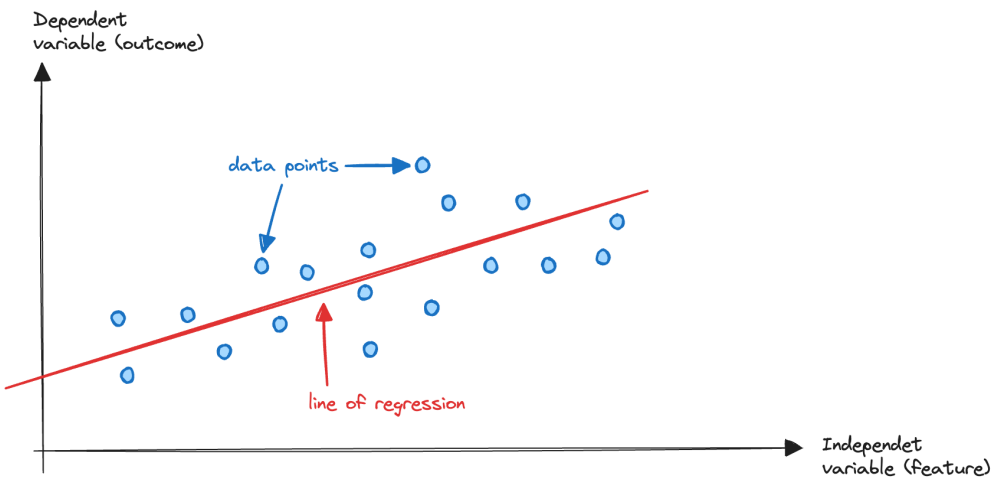
2026.03.04

Contents

1 Regression **1**
1.1 Get the data . . . 2
1.1.1 Load the data . . . 3
1.1.2 Show data . . . 5
1.1.3 Create a test set . . . 7
1.2 Discover and visualize data . . . 12
1.2.1 Visualize geographical information . . . 13
1.2.2 Looking for correlations . . . 14
1.2.3 Attribute combinations . . . 17
1.3 Prepare data . . . 18
1.3.1 Data cleaning . . . 19
1.3.2 Handling text and categorical attributes . . . 20
1.3.3 Custom transformers . . . 22
1.3.4 Feature scaling . . . 23
1.3.5 Pipelines . . . 25
1.4 Select and train a model . . . 26
1.4.1 Training . . . 26
1.4.2 Cross-Validation . . . 29
1.4.3 Save the model . . . 32
1.4.4 Fine-Tuning . . . 32
1.4.5 Evaluation . . . 36
1.5 Exercises . . . 37

1 Regression

Regression is a method for understanding the relationship between independent variables (**features**) and a dependent variable (**outcome**). Outcomes can then be predicted once the relationship between independent and dependent variables has been estimated. In regression problems, we will spend an large fraction of our time **acquiring, cleaning, and transforming data**.



1.1 Get the data

We use the California census data to build a model of housing prices in the state. This data includes metrics such as the population, median income, and median housing price for each block group in California. Block groups are the smallest geographical unit for which the US Census Bureau publishes sample data. We will call them "districts" for short. The model should learn from this data and be able to predict the median housing price in any district, given all the other metrics.

In typical environments our data would be available in a database and spread across multiple tables/documents/files. To access it, we first familiarize with the data schema. In this project, we will just download a single compressed file which contains a comma-separated values (CSV) file called housing.csv with all the data. It is always preferable to create a small function to download the file and decompress it. Having a function is useful, in particular if the data changes regularly: we can write a small script that uses the function to fetch the latest data. Automating the process of fetching the data is also useful if we need to install the dataset on multiple machines.

```
!mkdir data
```

```
mkdir: data: File exists
```

```
import urllib.request
import zipfile
```

```
def fetch_data(remote, local):
    urllib.request.urlretrieve(remote, local)
    with zipfile.ZipFile(local, 'r') as zip_ref:
        zip_ref.extractall('data/')

fetch_data("https://www.dropbox.com/scl/fi/2q1fsi1owt0607obsicjv"+
           "/housing.zip?rlkey=6yenwd0tusmvqeig997s03p8c&dl=1",
           "data/housing.tgz" )
```

1.1.1 Load the data

Now let's load the data using Pandas, in order to have a **DataFrame object** containing all the data. We can take a look at the top rows using the **head**() method.

```
import pandas as pd

data_set = pd.read_csv("data/housing.csv")
data_set.head()
```

```
   longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0    -122.23     37.88                41.0        880.0           129.0
1    -122.22     37.86                21.0       7099.0          1106.0
2    -122.24     37.85                52.0       1467.0           190.0
3    -122.25     37.85                52.0       1274.0           235.0
4    -122.25     37.85                52.0       1627.0           280.0

   population  households  median_income  median_house_value ocean_proximity
0       322.0       126.0         8.3252            452600.0        NEAR BAY
1      2401.0      1138.0         8.3014            358500.0        NEAR BAY
2       496.0       177.0         7.2574            352100.0        NEAR BAY
3       558.0       219.0         5.6431            341300.0        NEAR BAY
4       565.0       259.0         3.8462            342200.0        NEAR BAY
```

The **info**() method is useful to get a quick description of the data, in particular the total number of rows, each attribute's type, and the number of non-null values

```
data_set.info();
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

The **describe**() method shows a summary of the numerical attributes.

```
data_set.describe()
```

```
          longitude      latitude  housing_median_age   total_rooms  \
count  20640.000000  20640.000000        20640.000000  20640.000000
mean    -119.569704     35.631861           28.639486   2635.763081
std        2.003532      2.135952           12.585558   2181.615252
min     -124.350000     32.540000            1.000000      2.000000
25%     -121.800000     33.930000           18.000000   1447.750000
50%     -118.490000     34.260000           29.000000   2127.000000
75%     -118.010000     37.710000           37.000000   3148.000000
max     -114.310000     41.950000           52.000000  39320.000000

       total_bedrooms    population    households  median_income  \
count    20433.000000  20640.000000  20640.000000   20640.000000
mean       537.870553   1425.476744    499.539680       3.870671
```

```
std        421.385070   1132.462122    382.329753       1.899822
min          1.000000      3.000000      1.000000       0.499900
25%        296.000000    787.000000    280.000000       2.563400
50%        435.000000   1166.000000    409.000000       3.534800
75%        647.000000   1725.000000    605.000000       4.743250
max       6445.000000  35682.000000   6082.000000      15.000100

       median_house_value
count        20640.000000
mean        206855.816909
std         115395.615874
min          14999.000000
25%         119600.000000
50%         179700.000000
75%         264725.000000
max         500001.000000
```

To get information about categorical attribute, it is possible to use the **value_counts**() method

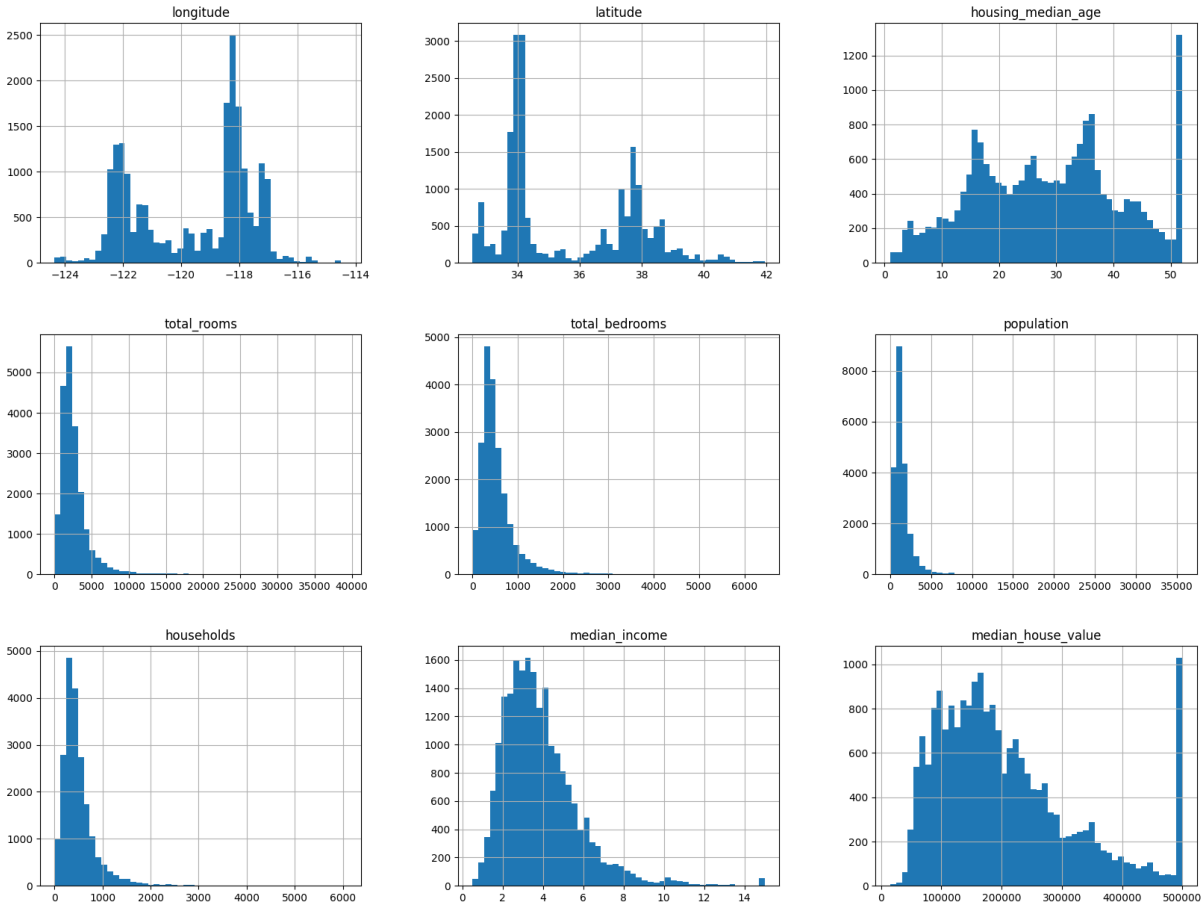
```
data_set["ocean_proximity"].value_counts()
```

```
ocean_proximity
<1H OCEAN     9136
INLAND        6551
NEAR OCEAN    2658
NEAR BAY      2290
ISLAND           5
Name: count, dtype: int64
```

1.1.2 Show data

A quick way to get a feel of the type of data we are dealing with is to **plot a histogram for each numerical attribute**. We can call the **hist**() method on the whole dataset.

```
data_set.hist(bins=50, figsize=(20,15));
```



There are a few things we might notice in the histograms:

- the "median income" does not look like it is expressed in dollars, probably numbers represent roughly tens of thousands of dollars, and data has been **scaled and capped** at 15 for higher median incomes.
- The "housing median age" and the "median house value" were also **capped**. The latter may be a serious problem since it is the **target attribute**. The algorithms may learn that prices never go beyond that limit. We have two options: collect proper labels, remove those districts from the dataset.
- Attributes have **very different scales**.
- Many histograms are **tail-heavy**, extend much farther to the right of the median than to the left. This may make it a bit harder for some ML algorithms to detect patterns.

Working with pre-processed attributes is common in ML, and it is not necessarily a problem, but we should **try to understand how the data was computed**.

1.1.3 Create a test set

A common problem in machine learning is **overfitting** (remember, a model that performs well on the training data, but generalizes poorly to any new data). This could involve learning noise in the data, or it could involve learning to identify specific inputs rather than whatever factors are actually predictive for the desired output. The other side of this is **underfitting** (remember, a model that doesn't perform well even on the training data), in that case typically we decide that our model isn't good enough and keep looking for a better one. Clearly, models that are too complex lead to overfitting and don't generalize well beyond the data they were trained on. So how do we make sure our models aren't too complex? The most fundamental approach involves using different data to train the model and to test the model.

Before we look at the data any further, we need to create a **test set**, put it aside, and **never look at it!** It may sound strange to voluntarily set aside part of the data at this stage. After all, we have only taken a quick glance at the data, and surely we should learn a whole lot more about it before we decide what algorithms to use. This is true, but **our brain is an amazing pattern detection system**, which means that it is highly prone to overfitting: if we look at the test set, we may notice interesting pattern in the test data that leads us to select a particular kind of Machine Learning model. This is called **data snooping bias**.

Creating a test set is theoretically simple: pick some instances randomly (typically 20%) of the dataset and set them aside.

```
import random

def split_data(data, prob):
    # Make a shallow copy
    data = data[:]
    # because shuffle modifies the list
    random.shuffle(data)
    # Use prob to find a cutoff
    cut = int(len(data) * prob)
    # and split the shuffled list there.
    return data[:cut], data[cut:]
```

```
train, test = split_data(data_set.values.tolist(), 0.80)

print(len(train))
print(len(test))
```

```
16512
```

```
4128
```

Often, we'll have paired input variables and output variables. In that case, we need to make sure to put corresponding values together in either the training data or the test data:

```
def train_test_split(xs, ys, prob):

    # Generate the indices and split them
    idxs = [i for i in range(len(xs))]

    train_idxs, test_idxs = split_data(idxs, prob)

    return ([xs[i] for i in train_idxs],  # x_train
            [xs[i] for i in test_idxs],   # x_test
            [ys[i] for i in train_idxs],  # y_train
            [ys[i] for i in test_idxs])   # y_test
```

If a model overfit the training data, then it will perform really poorly on the (completely separate) test data. Said differently, if it performs well on the test data, then you can be more confident that it's fitting rather than overfitting. This works, but it is not perfect: if we run the program again, it will generate a different test set! Over time, we will get to see the whole dataset, which is what we want to avoid. One solution is to **save the test set on the first run and then load it in subsequent runs**. Another option is to **set the random number generator's seed** so that it always generates the same shuffled indices. But these solutions will break the next time we fetch an updated dataset. To have a **stable train/test split** even after updating the dataset, a common solution is to use each instance's **identifier** to decide whether or not it should go in the test set. For example, we could compute a hash of each instance's identifier and put that instance in the test set if the hash is lower than or equal to 20% of the maximum hash value. This ensures that the test set will remain consistent across multiple runs, even if we refresh the dataset. The **Scikit-Learn** library provides a few functions to split datasets into multiple subsets in various ways. The simplest function is **train_test_split**().

```
from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(data_set,
                                       test_size=0.2,
                                       random_state=42)
```

```
len(train_set)
```

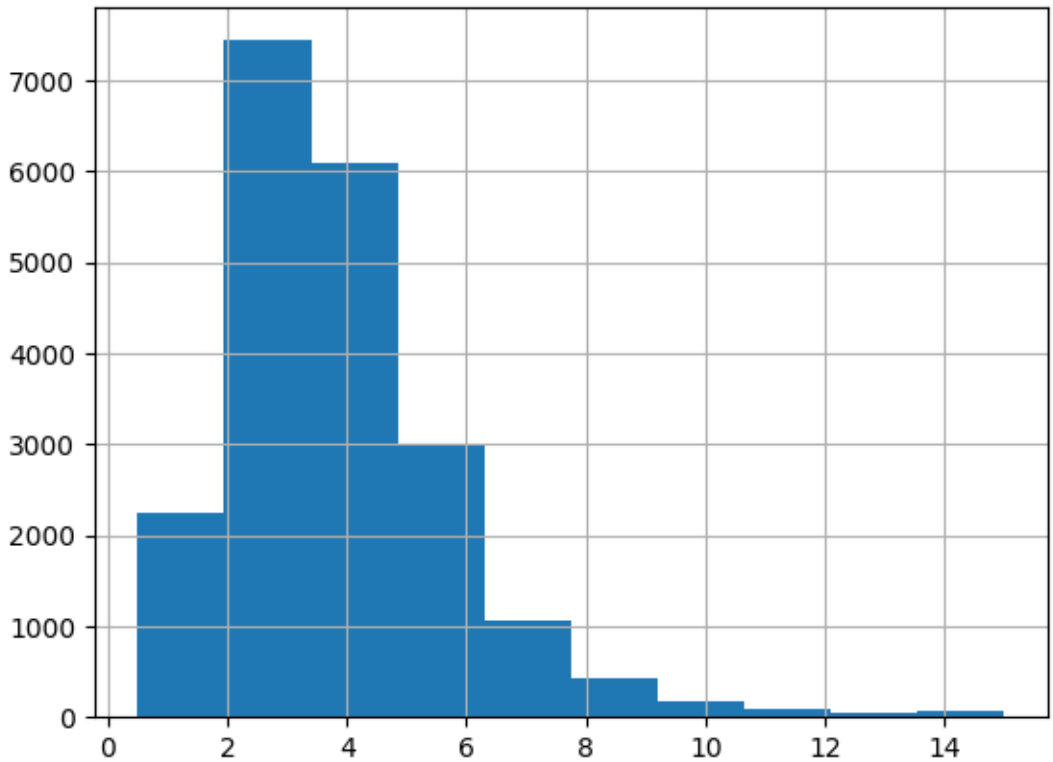
```
16512
```

```
len(test_set)
```

```
4128
```

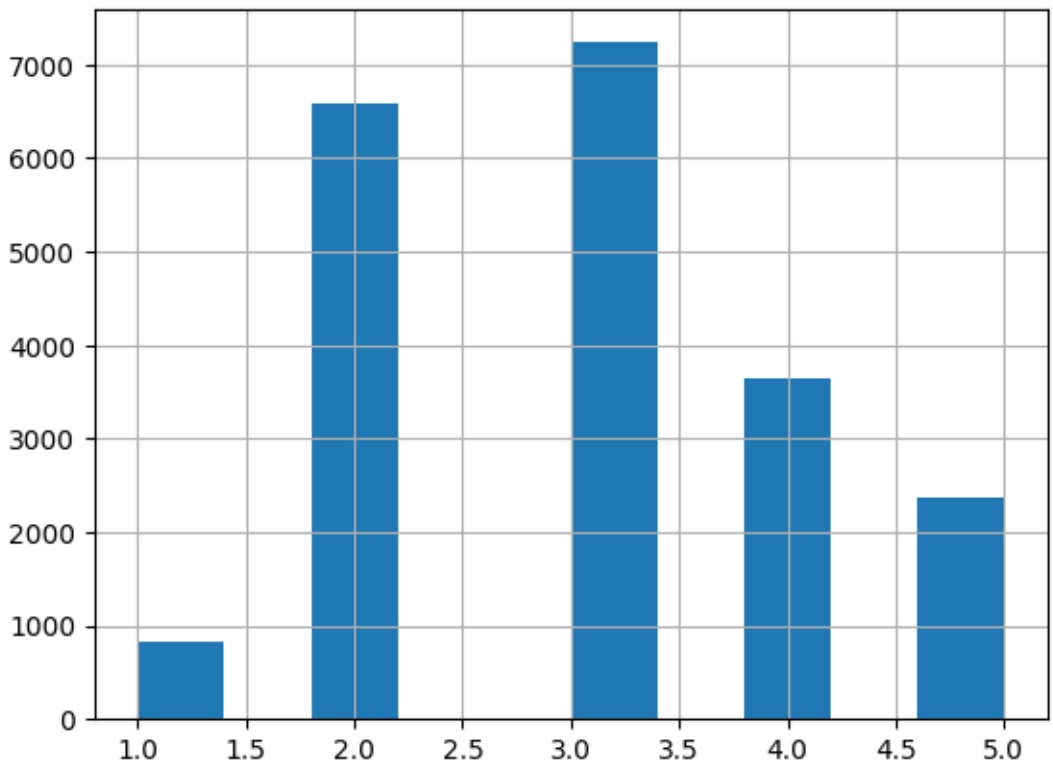
So far we have considered **purely random sampling methods**. This is generally fine if our dataset is **large enough**, but if it is not, we run the risk of introducing a significant **sampling bias**. Suppose that the "median income" is a very important attribute to predict median housing prices. We may want to ensure that **the test set is representative** of the various categories of incomes in the whole dataset. Since it is a continuous numerical attribute, we need to create a category attribute. Let's look at the histogram more closely.

```
data_set["median_income"].hist();
```



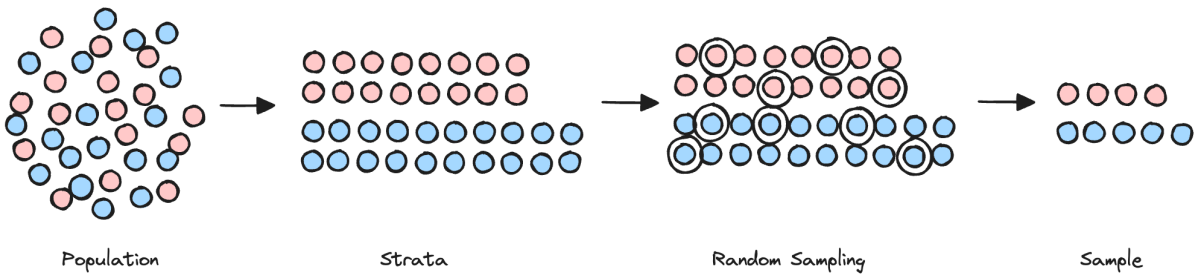
We can use the **cut**() method in order to create an "income" **category attribute** with five categories (labeled from 1 to 5).

```
import numpy as np

data_set["income_cat"] = pd.cut(data_set["median_income"],
                                bins=[0.,1.5,3.0,4.5,6.,np.inf],
                                labels=[1,2,3,4,5])

data_set["income_cat"].hist();
```



We can now use **stratified sampling** in order to **preserves the proportion of some category in the training and testing sets**. The population is divided into homogeneous subgroups called **strata**, and a number of instances **proportional to the size of the strata** are sampled from each stratum to guarantee that the test set is representative of the overall population:

$$\text{sample size} = \frac{\text{stratum size}}{\text{population size}} \times \text{dataset size}$$



We can use the Scikit-Learn's **StratifiedShuffleSplit** class. The following Python code splits the dataset into training and testing sets using the two different methods (Stratified Sampling and Random Sampling). After, the code calculates and shows the proportions of each category ("income_cat") in each dataset (stratified train/test sets, random train/test sets, and the original dataset).

```
import sklearn

# stratified sampling
split = sklearn.model_selection.StratifiedShuffleSplit(n_splits=1,
                                                       test_size=0.2,
                                                       random_state=42)

for train_index, test_index in split.split(data_set, data_set["income_cat"]):
    strat_train_set = data_set.loc[train_index]
    strat_test_set = data_set.loc[test_index]
```

```
# random sampling
random_train_set, random_test_set =
 ↪  sklearn.model_selection.train_test_split(data_set,
                                                                    test_size=0.2,
                                                                   random_state=42)
```

```
proportions = {
 'strat_test_set': strat_test_set["income_cat"].value_counts() / len(strat_test_set),
  ↪
 'strat_train_set': strat_train_set["income_cat"].value_counts() /
  ↪  len(strat_train_set),
 'data_set': data_set["income_cat"].value_counts() / len(data_set),
 'random_train_set': random_train_set["income_cat"].value_counts() /
  ↪  len(random_train_set),
 'random_test_set': random_test_set["income_cat"].value_counts() /
  ↪  len(random_test_set)
```

```
}

proportions_df = pd.DataFrame(data=proportions, index=[1, 2, 3, 4, 5])
proportions_df
```

```
   strat_test_set  strat_train_set  data_set  random_train_set  \
1        0.039971         0.039789  0.039826          0.039729
2        0.318798         0.318859  0.318847          0.317466
3        0.350533         0.350594  0.350581          0.348595
4        0.176357         0.176296  0.176308          0.178537
5        0.114341         0.114462  0.114438          0.115673

   random_test_set
1         0.040213
2         0.324370
3         0.358527
4         0.167393
5         0.109496
```

As we can see, the test set generated using stratified sampling has income category proportions almost identical to those in the full dataset, whereas the test set generated using purely random sampling is skewed. Now we should remove the income_cat attribute so the data is back to its original state.

```
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
```

1.2 Discover and visualize data

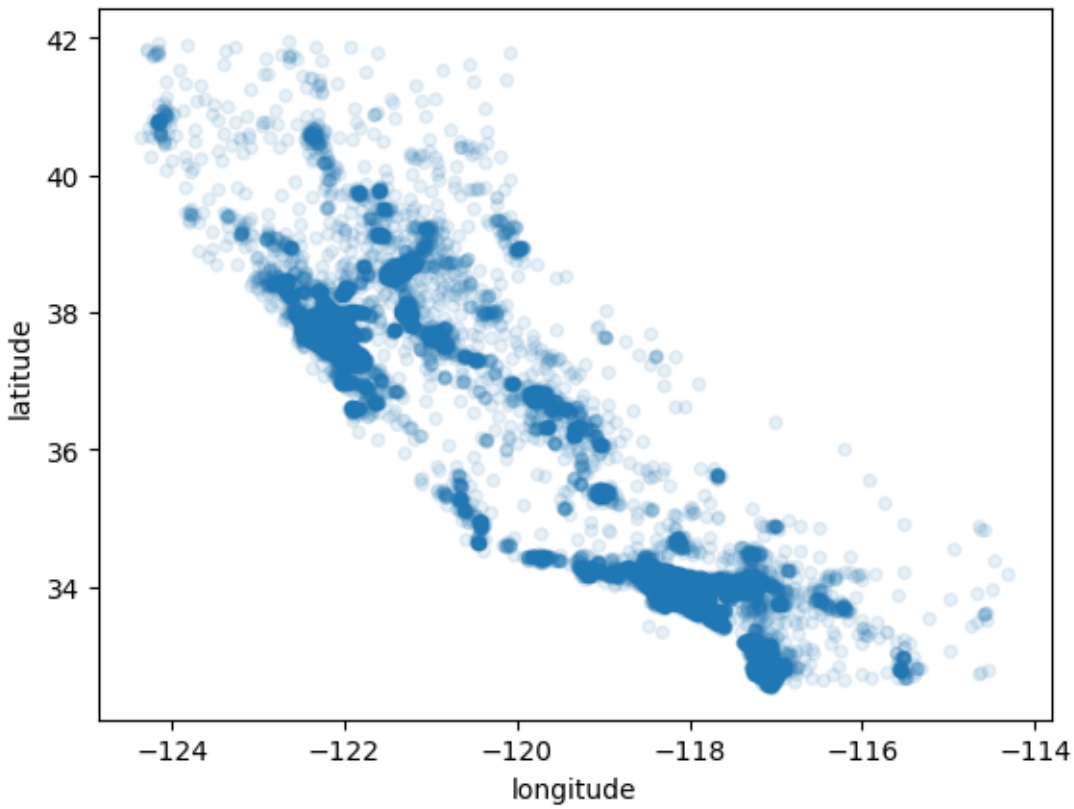
First, make sure we have put the test set aside and we are only exploring the training set. Also, if the training set is very large, we may want to sample an **exploration set**, to make manipulations easy and fast. In our case, the set is quite small, so we can just work directly on the full set. Let's create a copy so that you can play with it without harming the training set.

```
exploration_set = strat_train_set.copy()
```

1.2.1 Visualize geographical information

Since there is geographical information (latitude and longitude), it is a good idea to create a scatter plot of all districts to visualize the data. The **plot()** method from Pandas can create that scatter plot for us, in particular the parameter **alpha** can be user to modify the opacity of the points on the plot. A lower alpha value means the points are more transparent. This can be useful when there are many points; areas of the plot where points overlap will appear darker and can help to visualize areas of high density.

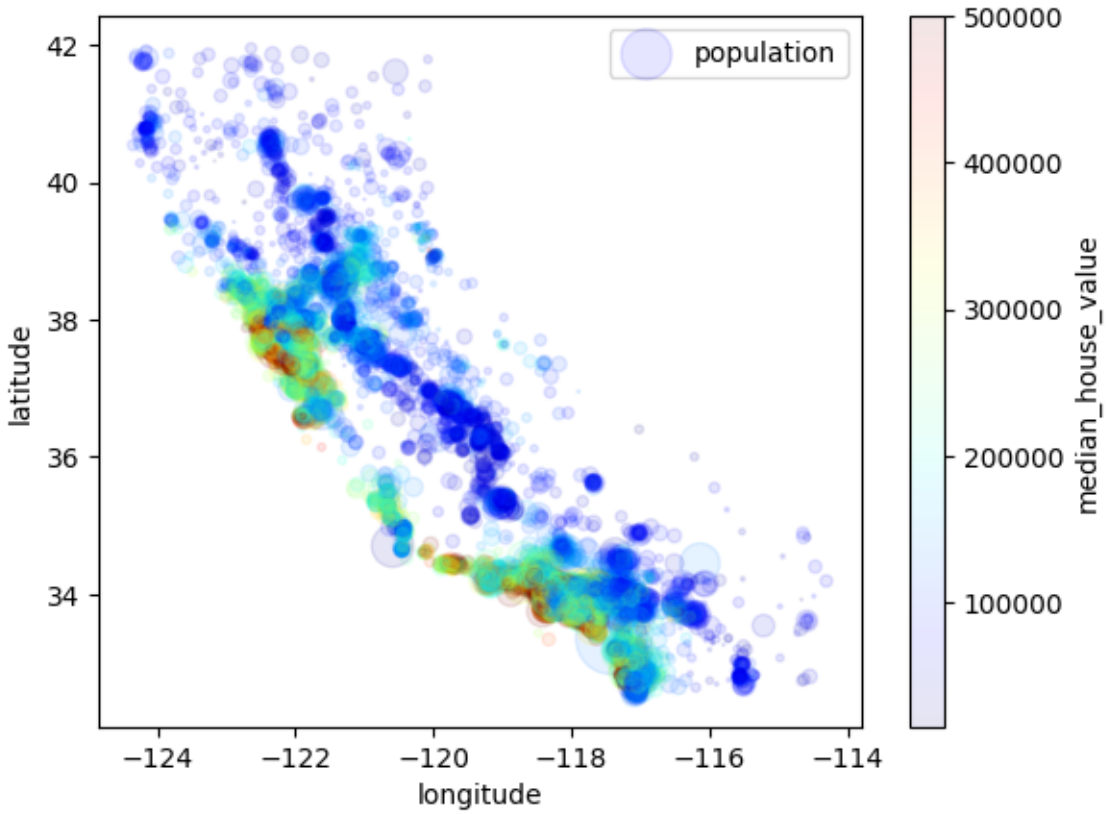
```
exploration_set.plot(kind="scatter", x="longitude", y="latitude", alpha=0.1);
```



You can use also other parameters to represent more dimensions of the data. For example, the **radius** of each circle could represent the district's population (option s), and the **color** could represent the price (option c). We will use a predefined color map (option cmap) called **jet**, which ranges from blue (low values) to red (high prices).

```
import matplotlib.pyplot as plt
```

```
exploration_set.plot(kind="scatter", x="longitude",  y="latitude", alpha=0.1,
                     s=exploration_set["population"]/50,
                     label="population",
                     c="median_house_value",
                     cmap=plt.get_cmap("jet"),
                     colorbar=True)
plt.legend();
```



The image tells that the housing prices are very much related to the location (e.g., close to the ocean) and to the population density. A **clustering algorithm** should be useful for detecting the main cluster and for adding new features that measure the proximity to the cluster centers.

1.2.2 Looking for correlations

Since the dataset is not too large, we can easily compute the **standard correlation coefficient** between every pair of attributes using the **corr()** method. The correlation coefficient is defined as follows:

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

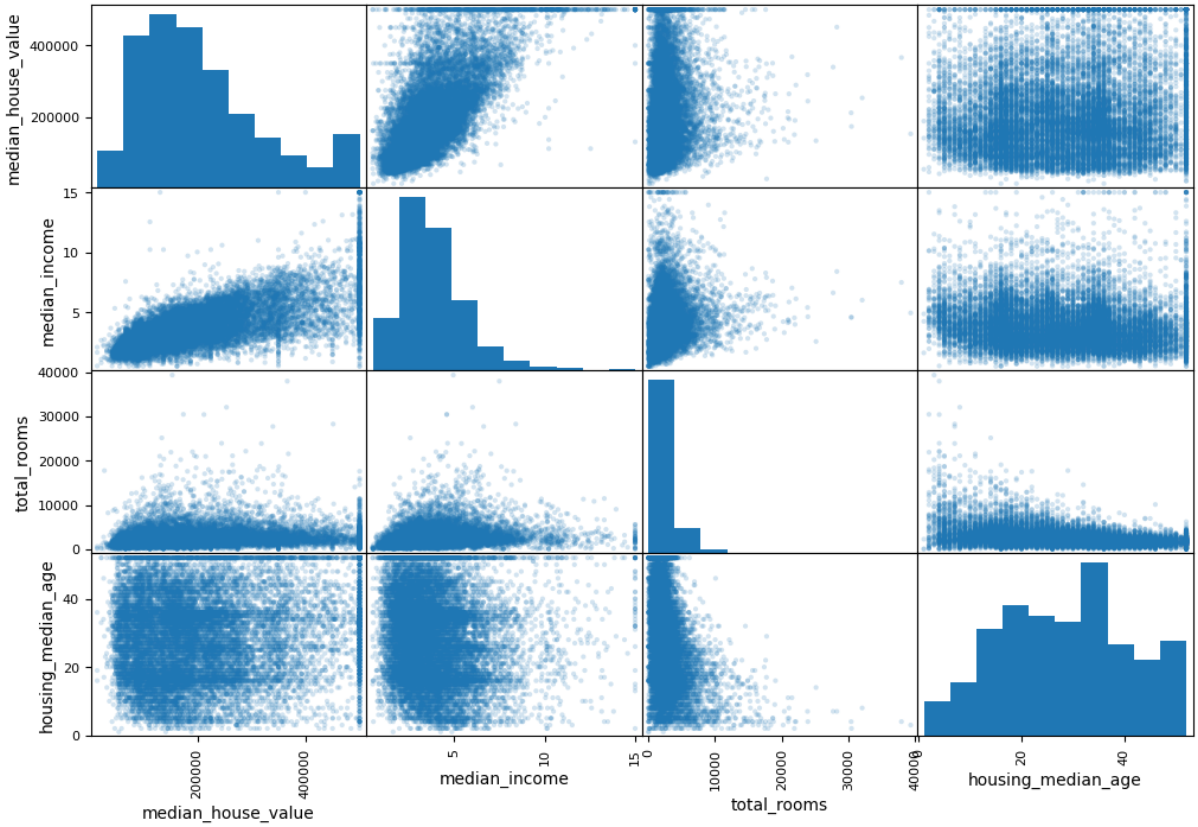
This coefficient ranges from –1 to 1. When it is close to 1, it means that there is a strong positive correlation; when the coefficient is close to –1, it means that there is a strong negative correlation; coefficients close to 0 mean that there is no linear correlation. The correlation coefficient **only measures linear correlations** ("if x goes up, then y generally goes up/down"). It may completely miss out on **non-linear relationships**.

```
corr_matrix = exploration_set.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value    1.000000
median_income         0.687151
total_rooms           0.135140
housing_median_age    0.114146
households            0.064590
total_bedrooms        0.047781
population           -0.026882
longitude            -0.047466
latitude             -0.142673
Name: median_house_value, dtype: float64
```

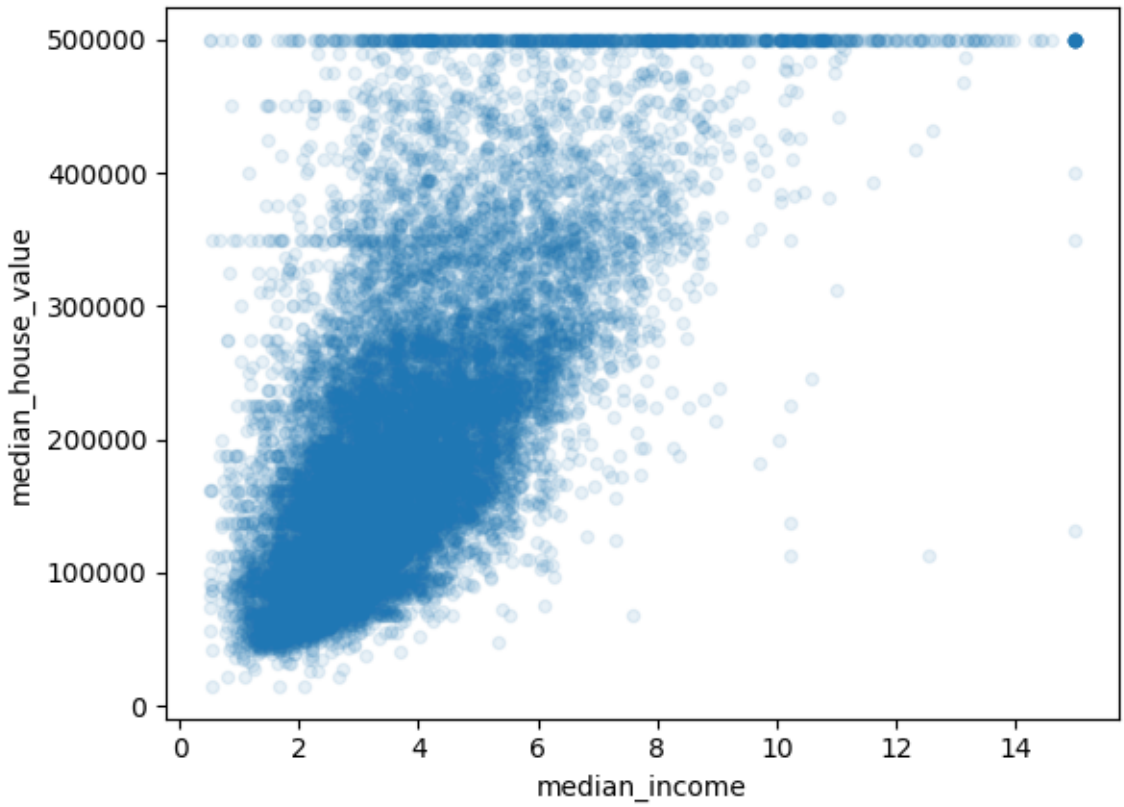
The "median house value" tends to go up when the "median income" goes up; we can see also a small negative correlation between the "latitude" and the "median house value". Another way to check for correlation between attributes is to use the **scatter_matrix**() function, which plots every numerical attribute against every other numerical attribute. We can focus on a few promising attributes that seem most correlated with the "median housing value". The main diagonal would be full of straight lines if pandas plotted each variable against itself, which would not be very useful. So instead pandas displays a histogram of each attribute.

```
pd.plotting.scatter_matrix(exploration_set[["median_house_value",
                                            "median_income",
                                            "total_rooms",
                                            "housing_median_age"]],
                           figsize=(12, 8),
                           alpha=0.2);
```



The most promising attribute to predict the "median house value" is the "median income"

```
exploration_set.plot(kind="scatter",
                     x="median_income", y="median_house_value",
                     alpha=0.1);
```



This plot reveals a few things:

- the correlation is very strong, you can clearly see the upward trend, and the points are not too dispersed,
- the price cap that we noticed earlier is clearly visible as a horizontal line at the top,
- the plot reveals other less obvious straight horizontl lines, we may want to try removing the corresponding districts to prevent our algorithms from learning to reproduce these data quirks.

1.2.3 Attribute combinations

One last thing we can do before preparing the data for ML algorithms is to try out various **attribute combinations**. For example, the "total number of rooms" in a district is not very useful if we don't know how many households there are. What we really want is the number of "rooms per household". Similarly, the "total number of bedrooms" by itself is not very useful: we probably want to compare it to the number of rooms. And the population per household also seems like an interesting attribute combination to look at. Let's create these new attributes.

```
exploration_set["rooms_per_household"] =
 ↪  exploration_set["total_rooms"]/exploration_set["households"]

exploration_set["bedrooms_per_room"] =
 ↪  exploration_set["total_bedrooms"]/exploration_set["total_rooms"]

exploration_set["population_per_household"] =
 ↪  exploration_set["population"]/exploration_set["households"]

corr_matrix = exploration_set.corr(numeric_only=True)
corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
median_house_value          1.000000
median_income               0.687151
rooms_per_household         0.146255
total_rooms                 0.135140
housing_median_age          0.114146
households                  0.064590
total_bedrooms              0.047781
population_per_household   -0.021991
population                 -0.026882
longitude                  -0.047466
latitude                   -0.142673
bedrooms_per_room          -0.259952
Name: median_house_value, dtype: float64
```

The new "bedrooms_per_room" attribute is much more correlated than the total number of rooms or bedrooms. Apparently houses with a lower bedroom/room ratio tend to be more expensive. The number of rooms per household is also more informative than the total number of rooms in a district—obviously the larger the houses, the more expensive they are. This round of exploration does not have to be absolutely exhaustive. The point is to start off on the right foot and quickly gain insights that will help to get a first reasonably good prototype. But this is an **iterative process**: once we get a prototype up and running, we can analyze its output to gain more insights and come back to the exploration step.

1.3 Prepare data

It's time to prepare the data for the machine learning algorithm. We should write functions for this purpose to allow us to reproduce these transformations easily on any dataset and to use

these functions in our live system to transform the new data before feeding it to our algorithms. First revert to a clean training set and separate the predictors and the labels.

```
predictors = strat_train_set.drop("median_house_value", axis=1)
labels = strat_train_set["median_house_value"].copy()
```

1.3.1 Data cleaning

ML algorithms cannot work with **missing features**. We have three options: delete the corresponding sample, remove the whole attribute or set the missing values to some value (zero, the mean, the median, etc.). We can accomplish these easily using **dropna**(), **drop**(), and **fillna**() methods.

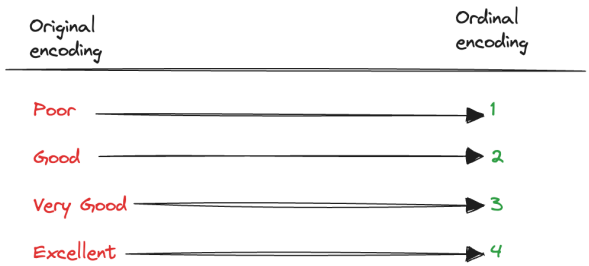
```
# first option
# predictors.dropna(subset=["total_bedrooms"])

# second option
# predictors.drop("total_bedrooms", axis=1)

# third option
# predictors["total_bedrooms"].fillna(predictors["total_bedrooms"].median())
```

Otherwise, Scikit-Learn provides a handy class to take care of missing values called **SimpleImputer**. We can specifying that we want to replace each missing values with the median of that attribute, however, since the median can only be computed on numerical attributes, we need to create a copy of the dataset without non-numerical attributes. The imputer computed the median of each attribute and stored the result in its statistics_ instance variable. In the example, we know than only "total_bedrooms" had missing values, but we cannot be sure that there won't be any missing values in new data after the system goes live, so it is safer to apply the imputer to all the numerical attributes.

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
predictors_num = predictors.drop("ocean_proximity", axis=1)
predictors_num = pd.DataFrame(imputer.fit_transform(predictors_num),
                              columns=predictors_num.columns,
                              index=predictors_num.index)
```

1.3.2 Handling text and categorical attributes

If an attribute doesn't contains arbitrary text, but there are a limited number of possible values, then this attribute is a **categorical attribute**. Most algorithms prefer to work with numbers, so we need to convert these categories from text to numbers.



It is possible to use the Scikit-Learn's **OrdinalEncoder**. The encoder stores the list of categories using the "categories_" instance variable. In the example, we have the "ocean_proximity" attribute.

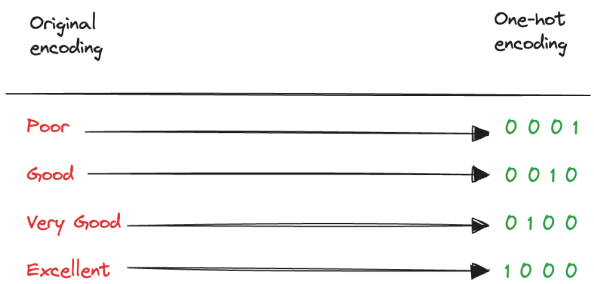
```
predictors_cat = predictors[["ocean_proximity"]]
print(predictors_cat[:10])
```

```
      ocean_proximity
12655          INLAND
15502      NEAR OCEAN
2908           INLAND
14053      NEAR OCEAN
20496       <1H OCEAN
1481         NEAR BAY
18125       <1H OCEAN
5830        <1H OCEAN
17989       <1H OCEAN
4861        <1H OCEAN
```

```
from sklearn.preprocessing import OrdinalEncoder

ordinal_encoder = OrdinalEncoder()
predictors_cat_encoded = ordinal_encoder.fit_transform(predictors_cat)
print(predictors_cat_encoded[:10])
print(ordinal_encoder.categories_)
```

```
[[1.]
 [4.]
 [1.]
 [4.]
 [0.]
 [3.]
 [0.]
 [0.]
 [0.]
 [0.]]
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],
      dtype=object)]
```

One issue with this representation is that algorithms can assume that two **nearby values are more similar than distant values**, because numbers are ordered. This may be fine in some cases (e.g., for ordered categories such as "bad," "average," "good," and "excellent"). It is not the case for the "ocean_proximity" column. To fix this issue, a common solution is the **one-hot encoding** that create one binary attribute per category (attributes equal to 1 for the corresponding category and 0 otherwise):



Scikit-Learn provides the **OneHotEncoder** class. The output is a **sparse matrix** (the matrix is full of 0s except for a single 1 per row, using memory mostly to store zeros; the sparse representation only stores the location of the nonzero elements). If we really want to convert it to a (dense array, just call the **toarray()** method. If a categorical attribute has a large number of possible categories, then one-hot encoding will result in a large number of input features. This may slow down training and degrade performance. If this happens, we can replace the categorical input with useful numerical features related to the categories: for example, we could replace the ocean_proximity feature with the distance to the ocean.

```
from sklearn.preprocessing import OneHotEncoder
```

```
cat_encoder = OneHotEncoder()
predictors_cat_1hot = cat_encoder.fit_transform(predictors_cat)
print(predictors_cat_1hot[:10])
predictors_cat_1hot.toarray()
```

```
  (0, 1) 1.0
  (1, 4) 1.0
  (2, 1) 1.0
  (3, 4) 1.0
  (4, 0) 1.0
  (5, 3) 1.0
  (6, 0) 1.0
  (7, 0) 1.0
  (8, 0) 1.0
  (9, 0) 1.0

array([[0., 1., 0., 0., 0.],
       [0., 0., 0., 0., 1.],
       [0., 1., 0., 0., 0.],
        ... ,
       [1., 0., 0., 0., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.]])
```

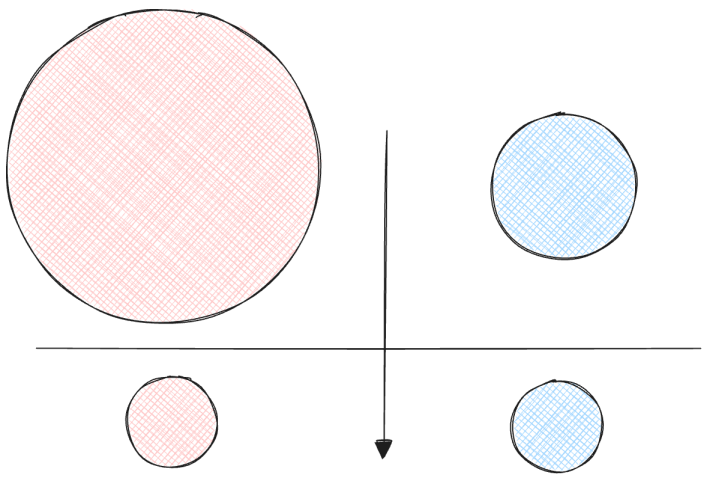
1.3.3 Custom transformers

Although Scikit-Learn provides many useful transformers, we will need to **write our own transformer** for tasks such as custom cleanup operations or combining specific attributes. To work seamlessly with Scikit-Learn, we need to create a class and implement three methods: **fit()**, **transform()** and **fit_transform()**. We can get the last one for free by simply adding **TransformerMixin** as a base class. If we add **BaseEstimator** as a base class we will also get two extra methods (get_params() and set_params()) that will be useful for automatic hyperparameter tuning. The transformer can have hyperparameters (it is often helpful to provide sensible defaults) to modify any data preparation step that we are not sure about: the more we automate data preparation steps, the more combinations we can automatically try out, making it much more likely that we will find a great combination (and saving a lot of time). For example, we create a transformer to add the combined attributes discussed earlier.

```
from sklearn.base import BaseEstimator, TransformerMixin

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):

    def __init__(self, add_bedrooms_per_room = True):
        self.add_bedrooms_per_room = add_bedrooms_per_room

    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        rooms_ix, bedrooms_ix, population_ix, households_ix = 3, 4, 5, 6
        rooms_per_household = X[:, rooms_ix] / X[:, households_ix]
        population_per_household = X[:, population_ix] / X[:, households_ix]
        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            return np.c_[X, rooms_per_household, population_per_household,
             ↪  bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]

attr_adder = CombinedAttributesAdder(add_bedrooms_per_room=False)
predictors_attribs = attr_adder.transform(predictors.values)
```

1.3.4 Feature scaling

ML algorithms don't perform well when the numerical attributes have **very different scales** (for example, features with larger magnitudes may dominate the learning process, biasing the model towards those features). Feature scaling is a preprocessing techniques used to standardize the range or distribution of features. The techniques ensure that features are on a similar scale, allowing machine learning models to perform optimally.



There are two common ways to get all attributes to have the same scale: **normalization** and **standardization**. In normalization values are shifted and rescaled so that they end up ranging from 0 to 1. We do this by subtracting the min value and dividing by the max minus the min:

$$X_{\text{normalized}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Standardization subtracts the mean value and then it divides by the standard deviation so that the resulting distribution has unit variance:

$$X_{\text{stardardized}} = \frac{X - \mu}{\sigma}$$

Unlike normalization, standardization does not bound values to a specific range, which may be a problem for some algorithms; however, it is much less affected by **outliers**. Scikit-Learn provides a transformers called **MinMaxScaler** and **StandardScaler** for this.

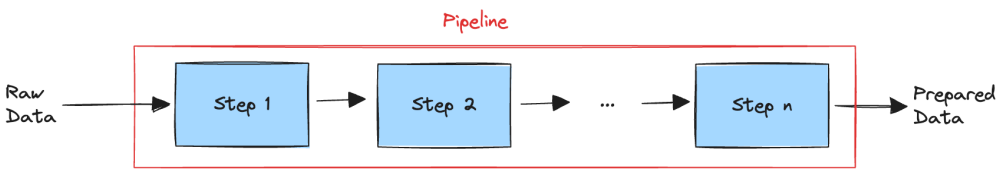
```
from sklearn.preprocessing import MinMaxScaler

scaling_encoder = MinMaxScaler()
predictors_scaled = scaling_encoder.fit_transform(predictors_num)
print(predictors_scaled[:10])
```

```
[[0.28784861 0.63549416 0.54901961 0.09836191 0.12806057 0.06261386
  0.13144137 0.11542599]
 [0.70916335 0.05844846 0.11764706 0.13516813 0.13740335 0.05639172
  0.14301718 0.40257376]
 [0.52888446 0.30074389 0.84313725 0.0410032  0.0496134  0.01861039
  0.05563854 0.16379774]
 [0.71912351 0.02231668 0.45098039 0.04759119 0.08327964 0.02508478
```

```
  0.08980583 0.11906732]
 [0.562749   0.18490967 0.50980392 0.0897899  0.10373711 0.05140279
  0.10791636 0.27561689]
 [0.23007968 0.575983   0.52941176 0.03054891 0.04027062 0.02020797
  0.04667662 0.22044524]
 [0.2310757  0.50903294 0.43137255 0.10721372 0.10776418 0.04986126
  0.11930545 0.48181404]
 [0.60159363 0.17640808 0.68627451 0.04288549 0.04204253 0.02172146
  0.05153099 0.31631288]
 [0.23904382 0.50265675 0.31372549 0.03367757 0.04333119 0.01135122
  0.04779686 0.08394367]
 [0.60458167 0.15727949 0.54901961 0.01294704 0.03656572 0.07531041
  0.0401419  0.        ]]
```

1.3.5 Pipelines

Scikit-Learn provides the **Pipeline** class to help with the **sequences of transformations**. The constructor takes a list of estimators defining a sequence of steps. All the estimators must be transformers (must have a fit_transform() method). When we call the pipeline's fit() method, it calls fit_transform() sequentially on all transformers, passing the output of each call as the parameter to the next call until it reaches the final estimator. The pipeline exposes the same methods as the final estimator.




```
from sklearn.pipeline import Pipeline

num_pipeline = Pipeline([('imputer', SimpleImputer(strategy="median")),
                         ('attribs_adder', CombinedAttributesAdder()),
                         ('std_scaler', MinMaxScaler()) ])

predictors_num_tr = num_pipeline.fit_transform(predictors_num)
```

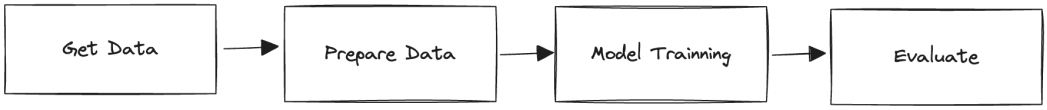
So far, we have handled the "categorical" columns and the "numerical" columns separately. It would be more convenient to have a single transformer able to handle all columns, applying the appropriate transformations to each column. Scikit-Learn introduced the **ColumnTransformer**

for this purpose. The constructor requires a list of tuples, where each tuple contains a name, a transformer, and a list of names of columns that the transformer should be applied to.

```
from sklearn.compose import ColumnTransformer

num_attribs = list(predictors_num)
cat_attribs = ["ocean_proximity"]

full_pipeline = ColumnTransformer([("num", num_pipeline, num_attribs),
                                   ("cat", OneHotEncoder(), cat_attribs)])

predictors_prepared = full_pipeline.fit_transform(predictors)
```

1.4 Select and train a model

We framed the problem, we got the data and explored it, we sampled a training set and a test set, and we wrote transformation pipelines to clean up and prepare data for ML. We are now ready to select and train a ML model. The whole process is summarized in the following figure:



1.4.1 Training

First, we train a Linear Regression model. This is were the magic happens, we don't know how the training works, and there different algorithms, however in order to use them we don't need to know the details, we at the moment can treat them as a black box. We just need to feed them the training data and they will learn to make predictions. Scikit-Learn provides a **LinearRegression** class to train a linear regression model using the **least squares** method. We can use the model to make predictions using the **predict**() method.

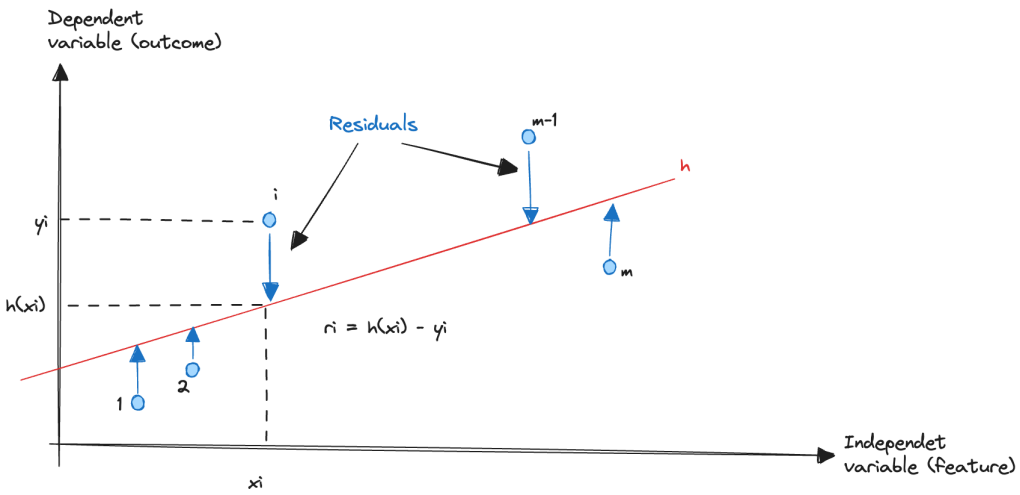
```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(predictors_prepared, labels);
```

Done! We now have a working Linear Regression model. Let's try it out on a few instances from the training set:

```
some_data = predictors.iloc[:5]
some_labels = labels.iloc[:5]

some_data_prepared = full_pipeline.transform(some_data)

print("Predictions:", lin_reg.predict(some_data_prepared))
print("Labels:", list(some_labels))
```

```
Predictions: [ 85657.90192014 305492.60737488 152056.46122456 186095.70946094
 244550.67966089]
Labels: [72100.0, 279600.0, 82700.0, 112500.0, 238300.0]
```

It works, although the predictions are not exactly accurate. Let's measure this regression model's **Root-Mean-Square Error (RMSE)** (a measure of the differences between the values predicted by the model and the real values, called **residuals**) on the whole training set:



$$\text{RMSE}(\mathbf{X}, h) = \sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(h(\mathbf{x}^{(i)}) - y^{(i)}\right)^2}$$

where $\mathbf{X}$ is the matrix containing all the feature values of all instances in the dataset, h is the model prediction function, and m is the number of instances in the dataset. Scikit-Learn provides the **mean_squared_error()** function.

```
from sklearn.metrics import mean_squared_error

predictions = lin_reg.predict(predictors_prepared)
lin_mse = np.sqrt(mean_squared_error(labels, predictions))
print("Linear model MSE: ", lin_mse)
```

```
Linear model MSE:  68627.87390018743
```

Clearly not a great score: most districts "median_housing_values" range between 120,000 dollars and 265,000 dollars, so a prediction error of 68,628 dollars is not very satisfying. This is an example of a model **underfitting** the training data. When this happens it can mean that the features do not provide enough information to make good predictions, or that the model is not powerful enough. The main ways to fix underfitting are to select a more powerful model, to feed the training algorithm with better features, to reduce the constraints on the model. Let's try a more complex model, a **Decision Tree**. Also in this case, we treat it as a black box, we don't know how Decition Tree works, we just use it. The **DecisionTreeRegressor** class is capable of finding complex nonlinear relationships in the data. The following code trains a Decision Tree model, just like we did for the Linear Regression model:

```
from sklearn.tree import DecisionTreeRegressor

tree_reg = DecisionTreeRegressor()
tree_reg.fit(predictors_prepared, labels)
```

```
DecisionTreeRegressor()
```

```
predictions = tree_reg.predict(predictors_prepared)
tree_mse = np.sqrt(mean_squared_error(labels, predictions))
print("Decision Tree model MSE: ", tree_mse)
```

```
Decision Tree model MSE:  0.0
```

No error at all? Could this model really be absolutely perfect? It is much more likely that the model has **overfit** the data. How can we be sure? We don't want to touch the test set until we are ready to launch a model we are confident about, so we need to use part of the training set for training and part of it for **model validation**. Before, let's try one last model now: the **RandomForestRegressor**, it works by training many Decision Trees on random subsets of the features, then averaging out their predictions. Building a model on top of many other models is called **Ensemble Learning**, and it is often a great way to push ML algorithms even further.

```
from sklearn.ensemble import RandomForestRegressor

forest_reg = RandomForestRegressor()
forest_reg.fit(predictors_prepared, labels)
```

```
RandomForestRegressor()
```

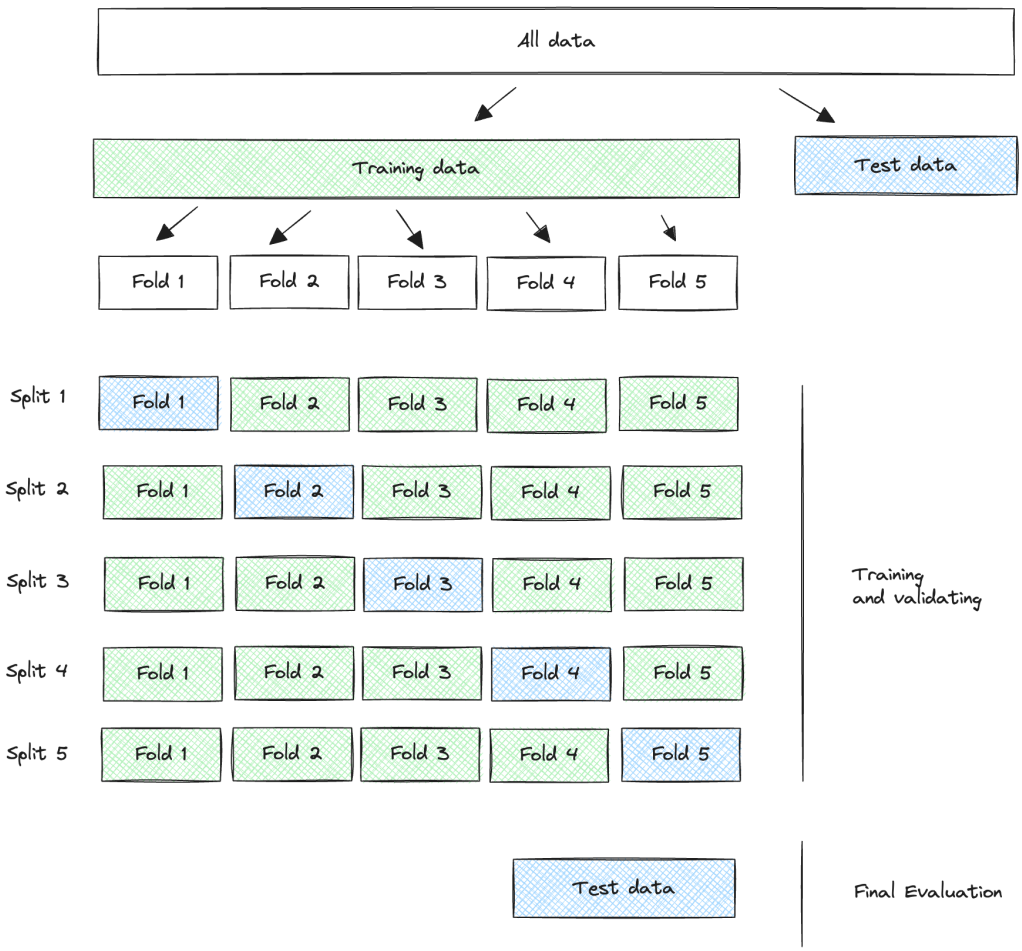
```
predictions = forest_reg.predict(predictors_prepared)
forest_mse = np.sqrt(mean_squared_error(labels, predictions))
print("Random Forest model MSE: ", forest_mse)
```

```
Random Forest model MSE:  18718.164430361317
```

Mmm...it looks like not so good as Decision Tree, but much better that Linear Regressor. We need more information on how to compare models.

1.4.2 Cross-Validation

We can use the **K-fold cross-validation** procedure. We randomly splits the training set into K distinct subsets (called folds), then we train and evaluate the model N times, picking a different fold for evaluation every time and training on the other (K-1) folds. The result is an array containing the N evaluation scores:



We can apply the cross-validation to the Decision Tree model:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(tree_reg, predictors_prepared,
                         labels, scoring="neg_mean_squared_error", cv=10)

tree_rmse_scores = np.sqrt(-scores)

print("Scores:", tree_rmse_scores)
print("Mean:", tree_rmse_scores.mean())
print("Standard deviation:", tree_rmse_scores.std())
```

```
Scores: [72198.58388536 70391.1684314  69287.78359818 72286.90490678
 70017.60660927 77428.38642126 70521.95935814 72349.93829109
 68732.93342826 70875.96002731]
```

```
Mean: 71409.12249570506
Standard deviation: 2328.4406379770126
```

Now the Decision Tree doesn't look as good as it did earlier. In fact, it seems to perform worse than the Linear Regression model. Notice that cross-validation allows us to get not only an estimate of the performance of our model, but also a measure of how precise this estimate is (standard deviation). Let's compute the same scores for the Linear Regression model just to be sure.

```
lin_scores = cross_val_score(lin_reg, predictors_prepared,
                             labels,
                             scoring="neg_mean_squared_error",  cv=10)

lin_rmse_scores = np.sqrt(-lin_scores)

print("Scores:", lin_rmse_scores)
print("Mean:", lin_rmse_scores.mean())
print("Standard deviation:", lin_rmse_scores.std())
```

```
Scores: [71762.76364394 64092.13546865 67747.81779114 68635.19072082
 66858.36456883 72528.03725385 73997.08050233 68802.33629334
 66437.34626364 70140.90243257]
Mean: 69100.19749391127
Standard deviation: 2885.0135897817336
```

The Decision Tree model is overfitting so badly that it performs worse than the Linear Regression model. What about the RandomForest?

```
forest_scores = cross_val_score(forest_reg, predictors_prepared,
                                labels,
                                scoring="neg_mean_squared_error",cv=10)

forest_rmse_scores = np.sqrt(-forest_scores)

print("Scores:", forest_rmse_scores)
print("Mean:", forest_rmse_scores.mean())
print("Standard deviation:", forest_rmse_scores.std())
```

```
Scores: [51423.11707782 49100.42673045 46746.38557403 52063.2787599
```

```
 47820.68711624 51612.54612542 52329.87092644 49720.83964514
 48708.18376839 53813.10594754]
Mean: 50333.8441671365
Standard deviation: 2136.0619352461017
```

This is much better, Random Forest looks very promising. However, note that the score on the training set is still much lower than on the validation sets, meaning that the model is still overfitting the training set. Possible solutions for overfitting are to simplify the model, constrain it (i.e., regularize it), or get a lot more training data. Before we dive much deeper into Random Forests, however, we should try out many other models from various categories of algorithms (e.g., **Support Vector Machines** or **Neural Networks**), without spending too much time tweaking the hyperparameters. The goal is to shortlist a few (two to five) promising models.

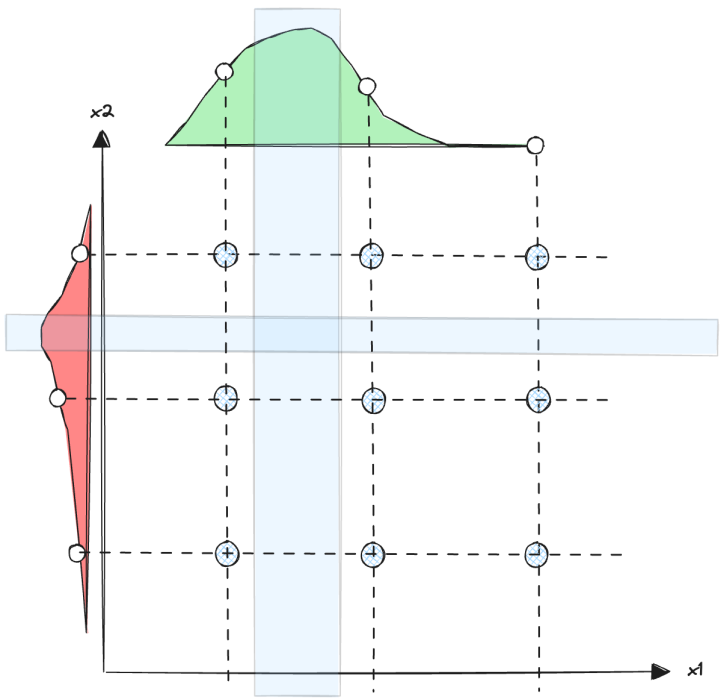
1.4.3 Save the model

We should save every model we experiment with so that we can come back easily to any model we want. Make sure to save both the hyperparameters and the trained parameters, as well as the cross-validation scores and perhaps the actual predictions as well. This will allow us to easily compare scores across model types, and compare the types of errors they make. We can easily save Scikit-Learn models by using the **joblib library**

```
import joblib

joblib.dump(forest_reg, "data/forest_reg.pkl")

# when we need it ...
my_model_loaded = joblib.load("data/forest_reg.pkl")
```

1.4.4 Fine-Tuning

Let's assume that we now have a shortlist of promising models, now we need to adjust (**fine-tune**) the hyperparameters. One option would be to fiddle with the hyperparameters manually, until we find a great combination of values. This would be very tedious work, and we may not have time to explore many combinations. Instead, we can use **Grid Search** to search for us. All we need to do is tell it which hyperparameters we want to experiment with, what values to try out, and it will use cross-validation to **evaluate all the possible combinations** of hyperparameter values. The following figure shows the idea for two hyperparameters:



When we have no idea what value a hyperparameter should have, a simple approach is to **try out consecutive powers of 10** (or a smaller number if you want a more fine-grained search). It may take quite a long time, but when it is done we can get the best combination of parameters. For example, the following code searches for the best combination of hyperparameter values for the RandomForestRegressor. Don't worry about what the hyperparameters mean for now (they will be explained in a Jupyter file focused on RandomForest).

```
from sklearn.model_selection import GridSearchCV

param_grid = [ { 'n_estimators': [3, 10, 30],
                 'max_features': [2, 4, 6, 8]
               },
               {
                 'bootstrap': [False],
                 'n_estimators': [3, 10],
                 'max_features': [2, 3, 4]
               }
              ]

forest_reg = RandomForestRegressor()

grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
```

```
                         return_train_score=True)

grid_search.fit(predictors_prepared, labels)
```

```
GridSearchCV(cv=5, estimator=RandomForestRegressor(),
             param_grid=[{'max_features': [2, 4, 6, 8],
                          'n_estimators': [3, 10, 30]},
                         {'bootstrap': [False], 'max_features': [2, 3, 4],
                          'n_estimators': [3, 10]}],
             return_train_score=True, scoring='neg_mean_squared_error')
```

Then we can get the best combination of parameters:

```
grid_search.best_params_
```

```
{'max_features': 8, 'n_estimators': 30}
```

we can also get the best estimator directly:

```
grid_search.best_estimator_
```

```
RandomForestRegressor(max_features=8, n_estimators=30)
```

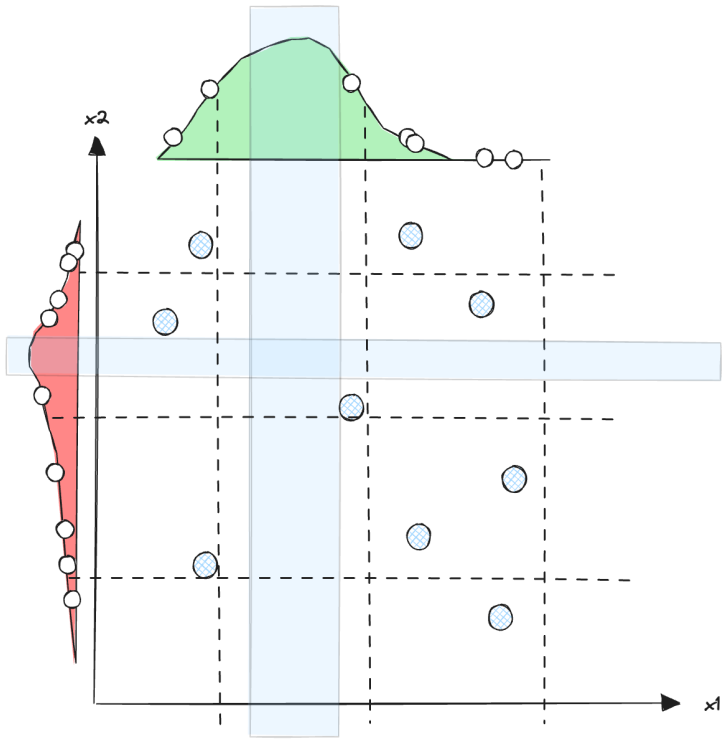
and of course the evaluation scores are also available:

```
cvres = grid_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(np.sqrt(-mean_score), params)
```

```
63691.40748055939 {'max_features': 2, 'n_estimators': 3}
54978.932016743616 {'max_features': 2, 'n_estimators': 10}
53054.100357062765 {'max_features': 2, 'n_estimators': 30}
60583.12097824862 {'max_features': 4, 'n_estimators': 3}
52851.3560658135 {'max_features': 4, 'n_estimators': 10}
50573.86392576462 {'max_features': 4, 'n_estimators': 30}
60086.78467371651 {'max_features': 6, 'n_estimators': 3}
```

```
52240.3278755907 {'max_features': 6, 'n_estimators': 10}
50126.98699742163 {'max_features': 6, 'n_estimators': 30}
59000.99454100527 {'max_features': 8, 'n_estimators': 3}
52212.87420273729 {'max_features': 8, 'n_estimators': 10}
49984.034062757404 {'max_features': 8, 'n_estimators': 30}
62706.740923801546 {'bootstrap': False, 'max_features': 2, 'n_estimators': 3}
54159.09303726247 {'bootstrap': False, 'max_features': 2, 'n_estimators': 10}
59031.758011518075 {'bootstrap': False, 'max_features': 3, 'n_estimators': 3}
52573.626732480414 {'bootstrap': False, 'max_features': 3, 'n_estimators': 10}
58291.0118945826 {'bootstrap': False, 'max_features': 4, 'n_estimators': 3}
52013.22476319237 {'bootstrap': False, 'max_features': 4, 'n_estimators': 10}
```

The grid search approach is fine when we are exploring relatively few combinations, but when the hyperparameter search space is large, it is often preferable to use **Randomized Grid Search** instead. It works in much the same way as the Grid Search, but instead of trying out all possible combinations, it **evaluates a given number of random combinations** by selecting a random value for each hyperparameter at every iteration.



This approach has two main benefits: if we let the randomized search run for, say, 1,000 iterations, this approach will explore 1,000 different values for each hyperparameter (instead of just a few values per hyperparameter with the grid search approach); by setting the number of iter-

ations, we have more control over the computing budget we want to allocate to hyperparameter search.

1.4.5 Evaluation

After tweaking our models for a while, we eventually have a system that performs sufficiently well. Now is the time to evaluate the final model on the test set. There is nothing special about this process; just get the predictors and the labels from the test set, run the full_pipeline to transform the data and evaluate the final model on the test set.

```
final_model = grid_search.best_estimator_

X_test = strat_test_set.drop("median_house_value", axis=1)
y_test = strat_test_set["median_house_value"].copy()

X_test_prepared = full_pipeline.transform(X_test)

final_predictions = final_model.predict(X_test_prepared)
final_mse = mean_squared_error(y_test, final_predictions)
final_rmse = np.sqrt(final_mse)

print("Final RMSE: ", final_rmse)
```

```
Final RMSE:  48067.00330516069
```

In some cases, such an estimate of the generalization error will not be enough to convince us to launch in production. We might want to have an idea of **how precise this estimate is**. For this, we can compute a 95% confidence interval (a range of values that we can be 95% confident contains the true mean of the population) for the generalization error using **scipy.stats.t.interval**()

```
from scipy import stats
confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
np.sqrt(stats.t.interval(confidence, len(squared_errors) - 1,
                         loc=squared_errors.mean(),
                         scale=stats.sem(squared_errors)))
```

```
array([46025.10138349, 50025.6299922 ])
```

If we did a lot of hyperparameter tuning, the performance will usually be slightly worse than what we measured using cross-validation (because the system ends up fine-tuned to perform well on the validation data and will likely not perform as well on unknown datasets). It is not the case in this example, but when this happens **we must resist the temptation to tweak the hyperparameters to make the numbers look good on the test set**, because the improvements would be unlikely to generalize to new data.

1.5 Exercises

1 - Try a Support Vector Machine regressor (sklearn.svm.SVR), with various hyperparameters such as kernel="linear" (with various values for the C hyperparameter) or kernel="rbf" (with various values for the C and gamma hyperparameters). How does the best SVM predictor perform?

```
from sklearn.model_selection import GridSearchCV

param_grid = [ {
                 'kernel': ['linear'],
                 'C': [10., 30., 100., 300., 1000., 3000., 10000., 30000.0]
               },
               {
                 'kernel': ['rbf'],
                 'C': [1.0, 3.0, 10., 30., 100., 300., 1000.0],
                 'gamma': [0.01, 0.03, 0.1, 0.3, 1.0, 3.0]
               }
              ]

svm_reg = sklearn.svm.SVR()
grid_search = GridSearchCV(svm_reg, param_grid, cv=5,
 ↪  scoring='neg_mean_squared_error', verbose=1)
grid_search.fit(predictors_prepared, labels);
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
```

The best model achieves the following score (evaluated using 5-fold cross validation):

```
rmse = np.sqrt(-grid_search.best_score_)
print("Best model RMSE: ", rmse)
```

```
Best model RMSE:  71395.02067621883
```

That's much worse than the RandomForestRegressor. Let's check the best hyperparameters found:

```
print("Best parameters: ", grid_search.best_params_)
```

```
Best parameters:  {'C': 30000.0, 'kernel': 'linear'}
```

The linear kernel seems better than the RBF kernel. Notice that the value of C is the maximum tested value. When this happens you definitely want to launch the grid search again with higher values for C (removing the smallest values), because it is likely that higher values of C will be better.

2 - Try replacing GridSearchCV with RandomizedSearchCV

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import uniform

param_distribs = { 'kernel': ['linear', 'rbf'],
                   'C': uniform(20, 200000),
                   'gamma': uniform(0.01, 5.0) }
svm_reg = sklearn.svm.SVR()
rnd_search = RandomizedSearchCV(svm_reg, param_distributions=param_distribs,
                                n_iter=50, cv=5, scoring='neg_mean_squared_error',
                                verbose=1, random_state=42)
rnd_search.fit(predictors_prepared, labels)
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits

RandomizedSearchCV(cv=5, estimator=SVR(), n_iter=50,
             param_distributions={'C': <scipy.stats._distn_infrastructure.rv_continuous_fro
                          'gamma': <scipy.stats._distn_infrastructure.rv_continuous_frozen
                                       'kernel': ['linear', 'rbf']},
                   random_state=42, scoring='neg_mean_squared_error',
                   verbose=1)
```

The best model achieves the following score (evaluated using 5-fold cross validation):

```
rmse = np.sqrt(-rnd_search.best_score_)
print("Best model RMSE: ", rmse)
```

```
Best model RMSE:  58568.245575842746
```

Now this is much closer to the performance of the RandomForestRegressor (but not quite there yet). Let's check the best hyperparameters found:

```
print("Best parameters: ", grid_search.best_params_)
```

```
Best parameters:  {'C': 30000.0, 'kernel': 'linear'}
```

This time the search found a good set of hyperparameters for the RBF kernel. Randomized search tends to find better hyperparameters than grid search.